Hydrofrac: lineage-driven fault injection for Hydro

Hydrofrac is a fault-injection search for Hydro programs. It runs a program in the Hydro simulator, records why each run produced what it produced, and uses that record to choose the next faults to try. It ends in one of two ways: a set of faults within declared bounds under which the program's specification fails, with the schedule that shows it; or a certificate that no fault set within those bounds breaks any run it observed. It is Molly (Alvaro, Rosen, Hellerstein, SIGMOD 2015) with the Hydro simulator in place of Dedalus, and with one addition that the rest of this document is about: for the property Hydro's type system already states, the specification is not written. It is read off the type.

This document describes what exists. The development record, with the reasoning behind each decision, is `history-hydrofrac-development.md`.

1. The property

Hydro's type system attaches a consistency label to every collection. The label `EventualConsistency` on a collection at a cluster says: *every member of the cluster that is given the same inputs computes the same contents.* The type system proves this over the shape of every execution, by the algebra of the operators between the inputs and the collection (`broadcast_closed` fans out to every member, a `fold` with a commutative, idempotent combiner is order- and duplicate-insensitive, and so on). What it cannot see is whether every member *is* given the same inputs. Some executions are missing events: a sender crashed between two sends of one fan-out, a message on a lossy channel was dropped. In those executions the label's premise is false, and the label says nothing.

Hydrofrac tests the premise. It takes one concrete execution of the program (a fixed topology and a fixed input workload, chosen by the test), and searches the faults the program's own transports admit, within declared bounds, for one that leaves a correct member without an input the others have. If it finds none, every correct member in every explored run held every fact of the collection; if it finds one, it names the fault and the schedule.

The division of labour is this. The type system quantifies over all executions and speaks to shape. Hydrofrac quantifies over all faults within bounds and speaks to one execution. A certificate is a statement about that execution; the confidence that it generalizes to other inputs and larger clusters is the confidence a test gives, sharpened by the fact that the search over faults is exhaustive within the bounds rather than random. A counterexample is a statement about the program.

2. What the certificate says, exactly

The specification is generated from the label (`Spec::eventual_consistency`). It is Molly's "the good outcome persists", instantiated for the label's good outcome:

> Let `FACTS` be the contents of the collection at the end of a failure-free run. Under any fault set within bounds, every correct member holds every fact of `FACTS` that some correct member holds.

There is no pre-condition. If the failure-free run produces no facts, the search is vacuous and says so (`Outcome::Vacuous`). Under a fault set that leaves no correct member holding any fact (the sole producer crashed before its first send), nothing is required of anyone; such a run yields no clause and the search moves on. This is Molly's treatment, and it is what the label means: the premise is about inputs that *were* given to some correct member.

A certificate is:

- **Within the fault model the program declares.** Every `Network` operator is a fault point; the transport it names is its fault vocabulary. A `lossy_delayed_forever` channel admits omission of any message sent before the end of finite failures; a `fail_stop` channel admits only its sender's crash. Crashes are available for the nodes the test

registers as crashable and are counted against a crash budget.

- **Within bounds the test declares.** The crash budget; the end of finite failures (`EFF`), after which no new fault takes effect and the protocol has the rest of the run to converge; the horizon, which is however long the test body drives the run.
- **For the execution the test fixes.** The cluster sizes, the inputs, the timer firings.
- **Over the schedules explored.** Under `Exploration::Exhaustive` every batching and ordering of the simulator is explored for every hypothesis, and the certificate is a proof within the bounds. Under `Exploration::Fuzz { instances }` a sample is explored, and the result is reported as a fuzz result, not a certificate.

A certificate is *sound by the form of the specification*. The search's completeness argument is that a fault set which leaves every delivery in the derivation of the specification intact cannot falsify the specification. That is true of a view that is monotone in the deliveries, and "every correct member holds every fact of a fixed set" is monotone in the deliveries by construction. This is why `FACTS` is fixed from a probe run rather than quantified over whatever each run produces: "every member holds everything any member holds" over an open domain is *not* monotone (a fault that makes one member derive a fresh fact falsifies it without touching any derivation), and would need a syntactic analysis of the view to grade. No such analysis exists in hydrofrac; the generated specification does not need one.

Specifications the user writes by hand (section 7) do not get this guarantee. Their certificates are printed as **RELATIVE**: within the fault model, relative to the specification's form.

3. What it is not

The certificate is not an agreement claim, not a liveness claim, and not a claim about inputs or topologies other than the test's.

Agreement ("members hold the *same* contents") is the type system's, given the premise; hydrofrac discharges the premise and does not re-prove the conclusion. Liveness ("every member *eventually* holds every fact") is made vacuous by design: the run has a finite horizon, and the specification is evaluated at its end. A protocol that would have converged with more time is reported as violated under the body that gave it less, which is a statement about the body as much as the protocol; the test author chooses the horizon, and the counterexample shows what was still missing. Interleavings are the simulator's: hydrofrac chooses fault sets; the simulator chooses schedules under each.

4. How it works

4.1 The simulator and the fault point

The Hydro simulator compiles a flow into a single-process executable with every network channel modelled as a queue, every tick as an explicitly scheduled batch, and every external input and timer driven by the test body. It explores batchings and orderings either exhaustively or by sampling. Hydrofrac places a **filter** in front of every network send. The filter has two verbs:

- **Omission.** Drop this message, identified by channel, sender, destination and wire bytes. Every byte-identical copy on that channel is dropped while finite failures last, so a retransmission with the same payload is the same message. Available on lossy channels.
- **Crash.** `crash(node after m)`: the node performs every send up to and including `m`, and every later send is dropped; its external outputs are suppressed. Named by a message rather than a send count, so the fault means the same thing in every schedule, and so that it can split the fan-out of one broadcast (the sender dies between two sends of the same item). Available on nodes the test registers.

Both kinds lie before `EFF`. Nothing in the program registers fault points; the transports it already declares are the fault model.

4.2 Lineage

Every run records every execution (an async run or one execution of a tick), every send with its fate, every receive by the delivery it took, and which execution produced each output. The **derivation** of a specification's view is the backward closure from the executions that produced it. Four levels of precision are available:

Level	What a tick execution depends on
`Exec`	every input of this execution and of every earlier execution of the same tick
`Slice` (default)	as above, narrowed by a static slice of the program per sink
`Record`	record-level provenance, by an IR rewrite that threads a provenance set through every operator (`SimFlow::with_provenance`)
`Counterfactual`	counterfactual replay: each node is re-executed alone with one delivery removed at a time; a delivery is in the derivation iff the node stops producing the output without it

Coarser levels over-approximate the derivation, which costs hypotheses, not soundness. Where a finer level cannot handle a node (a closure the rewrite does not see through; a replay that does not reproduce the output), it falls back to the next coarser one for that node.

4.3 The search

Molly's loop. The failure-free run (the probe) fixes `FACTS` and yields the first derivation. Each derivation is a clause: a set of deliveries at least one of which any breaking fault set must kill, where omission kills a message and `crash(node after m)` kills every later send of that node. A hypothesis is a hitting set of the clauses so far, within the crash budget. Each hypothesis is run under the configured exploration; if the specification fails in some schedule, that is the counterexample; if it holds, the run's derivation is a new clause (the retransmission, the redundant path, the echo). The search ends when no hitting set exists within the bounds (a certificate), a counterexample is found, or the hypothesis budget is exhausted (`Outcome::Budget`).

4.4 The generated specification

`Spec` is a Hydro flow at a dedicated observer process, compiled with the program. The test names the collection the label is on (`spec.observe_cluster(collection)`) and the cluster's members (`spec.correct_members(&cluster, 0..n)`); hydrofrac feeds the observer, before each run, the members the hypothesis does not crash and the facts `FACTS` learned from the probe. `spec.eventual_consistency(observed, correct)` builds the view: the distinct `(member, fact)` pairs at correct members over `FACTS` are as many as the `FACTS` facts present times the correct members. The view is evaluated at the end of each run.

The observer is part of the compiled flow, so the specification's derivation is recorded like any other output's, and the same lineage machinery produces the clauses.

4.5 The premise report

Every search under a `Spec` begins by listing every point in the program at which an `EventualConsistency` label is minted (a labelled operator none of whose inputs is labelled), and for each, whether a network channel feeds it and whether this specification observes it:

```
EventualConsistency at reliable_broadcast.rs:155 (Cluster(loc2v1)): observed by this specification
EventualConsistency at reliable_broadcast.rs:156 (Cluster(loc2v1)): NOT OBSERVED: an undischarged premise (fed by a netw
EventualConsistency at reliable_broadcast.rs:142 (Cluster(loc2v1)): not observed (fed by local sources only: nothing to
```

A network-fed label the specification does not observe is a premise nobody has discharged. The report does not fail the search; a program with several labelled collections needs several specifications, and which one a given test discharges is the test's business. On the hand-written harnesses in the tree it flags the coordinator's `prepare` and `commit` broadcasts in two-phase commit and the client's write broadcast and ZooKeeper's view broadcast in the ISR replication harness: `broadcast_closed` premises those harnesses carry by declaring the senders non-crashable.

5. Using it

Nothing is required of the program. The test fixes an execution, names the collection, and drives the run:

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();

let delivered = reliable_broadcast_closed(items, &members);   // labelled EventualConsistency

let mut spec = Spec::new(&mut flow);
let correct = spec.correct_members(&members, 0..3);
let observed = spec.observe_cluster(delivered.weaken_consistency());
spec.eventual_consistency(observed, correct);

let compiled = flow.sim()
    .skip_consistency_assertions()      // the simulator does not yet check them
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

Ldfi::new()
    .level(Level::Record)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    })
```

The body is the execution: which inputs, which timers, how long. For a protocol whose convergence needs time (retransmission timers, gossip rounds, leader change), the body fires those timers after `run.end_of_finite_failures()` so the protocol has its window. The result prints as a counterexample with its faults and failing schedule, or as a certificate with its grade, hypothesis count and exploration.

`Strategy::Blind` runs the same loop over the same fault universe without lineage (hypotheses are the untried fault sets by increasing weight), as the ablation that measures what lineage buys; `Ldfi::fuzz_faults_with` samples fault sets at random.

6. Results

Every producer of the `EventualConsistency` label in the codebase, and every demonstration protocol of `hydro_std::ec_inference_demos` whose labelled output is a cluster-resident set, has been run under the generated specification with one crash allowed and every registered node crashable (`hydro_test/src/cluster/ec_ldfi.rs`, `ec_demos_ldfi.rs`). Hypothesis counts include the probe run.

Program	Result
`broadcast_closed`, process → cluster, `fail_stop`	**Violated**, 5 hypotheses: `crash(source after bcast[→1]:Item(2))`. The sender dies between two sends of one item.

Program	Result
`broadcast_closed`, process → cluster, `lossy_delayed_forever`	**Violated**, 2 hypotheses: `omit(bcast[→0]:Item(2))`. One lost message; nothing retransmits.
`broadcast_closed`, cluster → cluster, `fail_stop`	**Violated**, 5 hypotheses: `crash(source@0 after bcast[0→1]:Item(10))`.
`broadcast_live_from_process` (one hop over live membership)	**Violated**, 5 hypotheses: `crash(source after bcast[→1]:2)`.
`leader_merge_broadcast`	**Violated**, 2 hypotheses: `crash(leader after merged[→1]:(writer 0, 100))`. The demo's own documentation says its label assumes the leader survives; this is that sentence as a counterexample.
Echo reliable broadcast (`hydro_test` version)	**Certificate**, 10 hypotheses, exhaustive.
`reliable_broadcast_closed`, `reliable_broadcast_live`	**Certificate**, 10 each, exhaustive.
`uniform_reliable_broadcast_closed`, thresholds 1 and 2	**Certificate**, 28 each, 8 sampled schedules per hypothesis.
`g_set_gossip`, `g_set_gossip_live`	**Certificate**, 13 each, 8 sampled schedules.
`multi_paxos`, the learners' `learned` output (1 proposer, 3 acceptors, 3 learners)	**Certificate**, 24 hypotheses, 8 sampled schedules. The learners re-broadcast what they learn; the certificate is that echo cycle's. The proposer is held correct.

Every single-hop dissemination fails under one crash or one omission; every protocol with an echo or gossip cycle holds. The transports themselves (`networking/mod.rs`) justify granting the label by reasoning about the recipient ("a failed connection models the recipient as failed") or about delay ("dropped messages are indefinitely delayed"), and neither reasoning survives one sender crash or one lost message. The label is a promise about the program that uses the transport, and only a program that retransmits keeps it. That is the right division: the type system cannot know whether the program retransmits, and hydrofrac can.

Other labelled collections in the tree are not fault questions: `source_interval` on a cluster (a stream of `()`), keyed folds with algebraic proofs (preserve the label given the same inputs), and `collect_quorum` (a function of the response set). Of the remaining demos, `quorum` and `epoch_splice` are combinators with no transport of their own (EC in, EC out; their premise is whichever broadcast fed them), `synod` labels nothing `EventualConsistency`, and `abd`'s replica register is in the class but is a lattice `Singleton` the set-based specification cannot observe correctly (section 8).

7. Hand-written specifications

`Spec` also accepts `PRE` and `POST` views written in Hydro, and `Ldfi` accepts goals over plain outputs (`goal`, `goal_all_members`, `goal_witnessed`). These are how hydrofrac is run against properties the type system does not state: Raft's and Paxos's durability of committed entries, two-phase commit's atomicity, ISR replication's agreement, quorum's claim counting, retry-until-ack delivery (`raft_ldfi.rs`, `paxos_ldfi.rs`, `two_pc_ldfi.rs`, `isr_replication.rs`, `quorum_ldfi.rs`, `reliable_delivery.rs`). Their certificates are graded RELATIVE: no form argument grades them, and the user is told so.

They have found real bugs. An injected Raft election-restriction bug (index-only log comparison) is found in 90 hypotheses with four omissions and no crash, a scenario the author had not written down; correct Raft certifies in 235 under the same bounds (both tests `#[ignore]`d for running time: thirty minutes and an hour). Paxos's P1b quorum counted replies rather than acceptors, a defect of the production protocol exposed once the body's `EFF` moved past the first ballot; correct Paxos, the Paxos key-value store and compartmentalized Paxos certify in 372, 301 and 568 hypotheses (ignored, 45 minutes together). The simulator found two defects on the way that are not fault questions: `collect_quorum_with_response` was not a function of its input (its `nondet!` justification was false; fixed, with a test), and a `fold` with an idempotence proof had no simulator implementation of its retry-only cast (now the identity,

since the simulated network never duplicates).

8. Limitations

- **One execution.** The certificate is for the test's topology and inputs. Nothing in hydrofrac generalizes it; that is the test author's induction.
- **Sampled schedules are not certificates.** Protocols with large schedule spaces (uniform broadcast, gossip, multi-Paxos) are run under `Fuzz`, and reported as such.
- **The horizon is the body's.** A protocol is given the time the body gives it. An unconverged run is a counterexample, not vacuous; a convergence witness that the protocol reports (every correct member has quiesced on the same view) would make it vacuous, and is not built. Raft's hand-written `PRE` ("some member committed") is the case where this cost effort: its body was tuned until correct Raft converged.
- **Set-valued collections only.** The generated specification observes a stream of facts. A lattice `Singleton` (`abd`'s max-by-timestamp register) needs a variant that observes the final value per member and checks equality; not built.
- **Static membership.** The simulator has everyone joined from the start; the `_live` demos are tested for faults, not for join timing.
- **Lineage gaps.** The provenance rewrite does not handle `entries_partially_ordered` on a keyed stream (`leader_merge` runs on `Level::Slice`). Counterfactual replay can panic inside a program that assumes a delivery it is being denied; the panic is contained on the generated side of the dynamic-library boundary and treated as inconclusive for that node.
- **The premise report is a report.** It does not fail a test. Making it one needs a notion of "every search over this program", which a per-program registry of specifications could provide.
- **Byzantine, partition, and duplication faults** are outside the vocabulary. Partitions are expressible as omission sets on lossy channels and are found that way (the Raft counterexample is four omissions on two links); duplication has no fault point.

9. Next

In order of what the argument of section 1 needs:

1. The lattice-aware specification for `Singleton` outputs, and `abd` under it.
2. A convergence witness in the generated specification for protocols that report one, so that horizon is a parameter rather than a verdict.
3. Decision-trace replay: record every nondeterministic decision of a run and replay a prefix with one decision overridden. This is the general counterfactual oracle of which node-level replay is the coarse instance.
4. Raft's `committed` under the generated specification (the harness is written; the run is an hour).